Chatbot Application Development

Use Case: Chatbot Application – React/Basic HTML Frontend, Python Backend with Azure OpenAI

Objective:
To practice building a basic chatbot application using a React frontend/Basic HTML Frontend, a Python backend, and the Azure OpenAI API to generate responses.

Requirements:

1. **Frontend**
    - Create a simple webpage where users can type questions and see responses.
    - Include:
        - An input box where users type their questions.
        - A "Send" button to submit questions.
        - An area below to display responses.
    - Use React to manage the input and responses. (Hint: Try using "useState" for input and response states.) You can also use vanilla HTML and JS to build.
2. **Backend Basics (Python + Azure OpenAI)**
    - Set up a simple API in Python (using Flask or FastAPI).
    - Create one endpoint (/query) that receives a question from the frontend.
    - Set up a connection to the **Azure OpenAI API** to generate responses:
        - Use the Azure OpenAI API to process the question and generate a response.
        - Include your API keys securely in the code or use environment variables.
    - Add a simple check: if the user input is empty, return a message like "Please ask a question!"
3. **Testing the Chatbot**
    - Send a question from the frontend app to the Python backend, which then connects to the Azure OpenAI API, and show the response on the frontend.
4. **Instructions**
    - Create a short README file explaining documenting code.
    - Snippets/Videos of apps running.
    - Codefiles in a zip folder

5. **Evaluation Criteria:**
    - **Basic Functionality:** The chatbot can receive a question and display a response from Azure OpenAI.
    - **React Basics:** Shows understanding of React by managing inputs and responses.
    - **Python Basics:** Simple setup of an API to receive questions, connect to Azure OpenAI, and respond.
    - **Instructions:** Easy-to-follow instructions for setting up and testing.

6. **Bonus Challenge:**

    Add a clear button to erase the conversation history.

    Add a loading message (e.g., “Thinking…”) that appears while the backend is preparing a response.